# Image Compression and Classification with Deep Learning

EE413 – Applied Digital Signal Processing

Ahmad Al-Eid
Ali Alhajji
Mohammed Aldubais
Faisal Alsalhi

# Agenda

01 Introdcution

02 Model Selection and Baseline Establishment

03 Wavelet-based Image Compression

04 Fine-tuning on Compressed Data

# 01

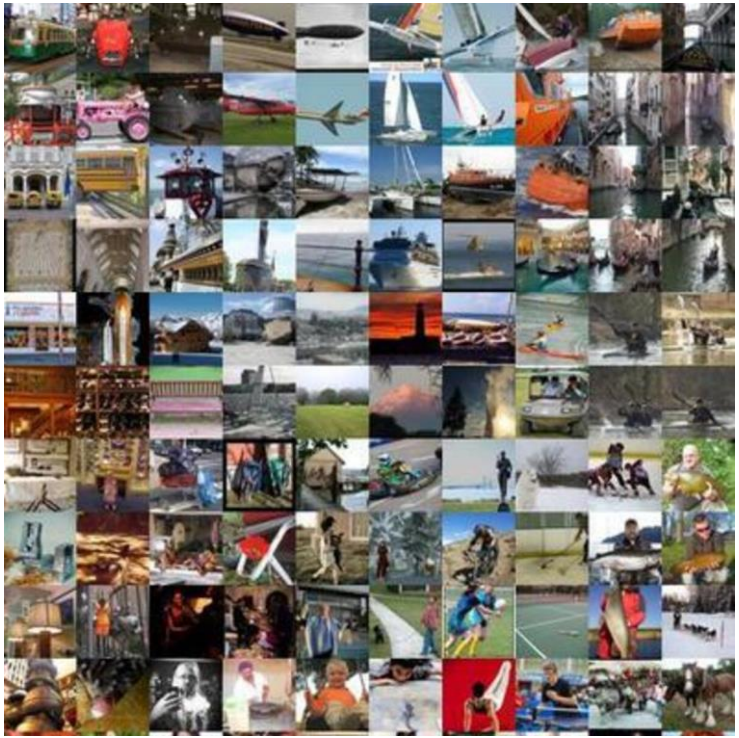
# Introdcution

# Team Work Split

- Model Selection and Fine Tunning (Baseline Establishment) / Ahmad Al-Eid
- Wavelet-based Image Compression / Mohammed Aldubais
- Fine-tuning on Compressed Data / Faisal Alsalhi & Ali Alhajji

# Dataset

- Subset of ImageNet with 100 classes
- Diverse object categories
- Sufficient to demonstrate compression effects



# 02

# Model Selection and Baseline Establishment

# Data Pipeline

- Resize images to 96 x 96
- Training transforms: flip + small rotation + normalization
- Split: 80% train, 10% val, 10% test

```
tfm_train = transforms.Compose([
    transforms.Resize((img_size, img_size)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=10),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),
])

tfm_eval = transforms.Compose([
    transforms.Resize((img_size, img_size)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),
])
```

# Model One Architecture and Training Process - ResNet18

- Strong and stable baseline model
- Loaded with ImageNet pre-trained weights
- Final layer replaced for 100 classes
- Advantage: higher accuracy and better feature learning
- Disadvantage: larger model with more parameters and slower training

# Model Two Architecture and Training Process - MobileNetV3

- Lightweight and efficient
- Fine-tuned with same pipeline for fair comparison
- Advantage: small size, faster training, fewer parameters
- Disadvantage: slightly lower accuracy than ResNet18

# Training Hyperparameters

- Loss: CrossEntropyLoss

- Optimizer: AdamW

- Learning rates: ResNet18: 0.0003,

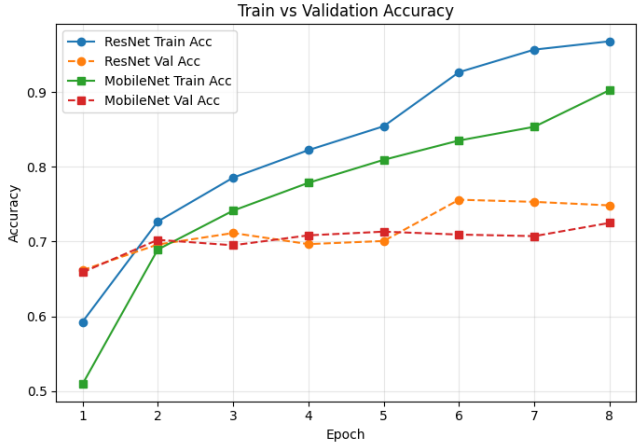
MobileNetV3: 0.0004

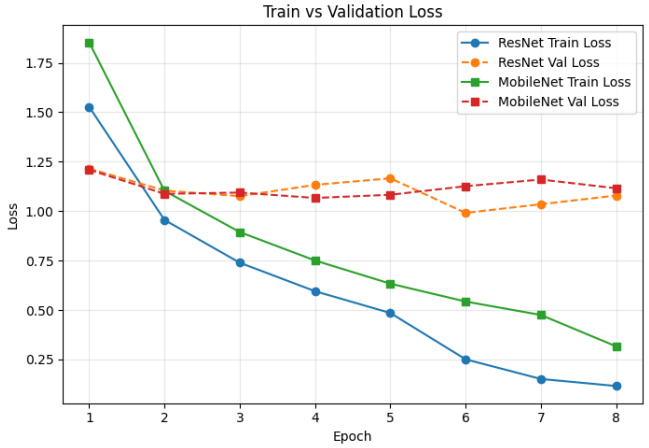
- Epochs: 8, Batch size: 64, Weight decay: 0.0001

```
epoch_budget = 8
lr_res = 0.0003
lr_mob = 0.0004
wd_common = 0.0001
stop_patience = 3
loss_oracle = nn.CrossEntropyLoss()
```

# Validation Strategy

- ResNet learned faster and reached higher training accuracy than MobileNet
- Training loss decreased for both models
- Validation accuracy improved early then plateaued around epochs 5–8
- Early stopping helped reduce overfitting



# Baseline Results

| Model | Parameters | Best Val Acc | Test Acc | Macro-F1 | Test Loss |
|---|---|---|---|---|---|
| ResNet18 | 11.21M | 0.756 | 0.747 | 0.747 | 0.987 |
| MobileNetV3-Small | 1.58M | 0.725 | 0.726 | 0.725 | 1.117 |

# Baseline Results

- ResNet18 is the best baseline performance on the uncompressed test data
- ResNet18 achieved higher accuracy (0.747) than MobileNetV3-Small (0.726)
- ResNet18 also achieved higher macro f1 (0.747) vs MobileNetV3-Small (0.725) so class balanced performance is better
- ResNet18 has lower test loss (0.987) compared to MobileNetV3-Small (1.117) so it has more confident and correct predictions
- Tradeoff: MobileNetV3-Small use fewer parameters (1.58M) than ResNet18 (11.21M) so it is more lightweight but less accurate

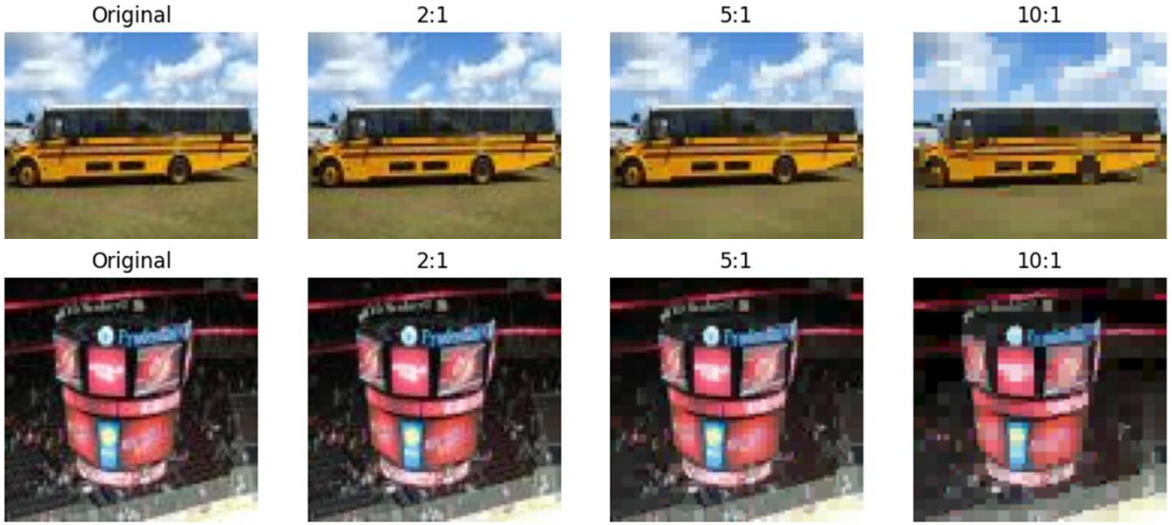
# 03

# Wavelet-based Image Compression

# Wavelet Compression Method

- Haar Wavelet was applied to the images (RGB images).
- Compression was done by hard thresholding. (high magnitude coefficients were kept and low magnitude coefficients were discarded)
- The compression ratios used are: 2:1, 5:1, 10:1.

# Visual Comparison




As the compression ratio increase, the minute details get lost

# Image Quality Results

| Compression | Coefficients kept | Average PSNR | Average SSIM |
|---|---|---|---|
| 2:1 | 50% | 47.605 | 0.9938 |
| 5:1 | 20% | 35.039 | 0.9435 |
| 10:1 | 10% | 29.336 | 0.8588 |

PSNR decreased from 47.605 at 2:1 to 29.336 at 10:1, while SSIM decreased from 0.9938 to 0.8588.

# Classification Results

| Test Set | Coefficients kept | Accuracy | Macro F1 | Accuracy Drop |
|---|---|---|---|---|
| Original | 100% | 74.71% | 74.73% | 0.00% |
| Wavelet 2:1 | 50% | 74.48% | 74.51% | 0.23% |
| Wavelet 5:1 | 20% | 71.90% | 72.04% | 2.81% |
| Wavelet 10:1 | 10% | 62.53% | 62.81% | 12.19% |

ResNet18 accuracy stayed almost unchanged at 2:1 compression but dropped by 12.19% at 10:1 compression.

# Most Affected Classes

The most affected classes lost between 26% and 41% accuracy at 10:1 compression.

| Class | Original Acc. | 10:1 Acc. | Drop |
|---|---|---|---|
| n02966193 | 80.00% | 38.33% | 41.67% |
| n02120079 | 80.00% | 48.33% | 31.67% |
| n04067472 | 53.33% | 25.00% | 28.33% |
| n02457408 | 70.00% | 41.67% | 28.33% |
| n02113712 | 61.67% | 35.00% | 26.67% |

Wavelet compression provides a trade-off between image size and classification performance. Moderate compression preserved most classification accuracy, while aggressive compression removed important visual details and caused significant accuracy degradation.

# 04

# Fine-tuning on Compressed Data

1

PART 3

# Fine-Tuning Model One (ResNet-18)

Approach

*Ali Alhajji*

## Approach & Setup

### Starting Point

Warm-started from the Part 1 ResNet-18 checkpoint (resnet_star) — not re-initialized from ImageNet. This preserves Mini-ImageNet knowledge and makes fine-tuning faster and more stable.

### Training Data

Medium compression (5:1, threshold = 0.15) applied to the training set via DWT + hard thresholding. Chosen as a balance between too mild (2:1) and too destructive (10:1).

### Hyperparameters

AdamW · lr = 5e-5 · weight decay = 1e-4 · batch = 64. Lower LR than Part 1 to prevent catastrophic forgetting. Up to 10 epochs with patience-3 early stopping.

2

# Training Results & Performance

Ali Alhajji · ResNet-18 · Part 3

| 10 | 5e-5 | 5:1 |
| --- | --- | --- |
| **Epochs Budget** | **Learning Rate** | **Training Compression** |
| Early stopped at best val | AdamW + ReduceLROnPlateau | Medium threshold = 0.15 |

## Training Loop Design

1. Each epoch: train on wavelet-compressed images → record batch loss
2. Validation on compressed val set → ReduceLROnPlateau adjusts LR on plateau
3. Early stopping (patience = 3) → best checkpoint restored after training
4. Evaluation: accuracy + macro F1 reported across all 3 compression levels

# Comparison & Key Findings

**Model Comparison — Accuracy Drop vs. Uncompressed Baseline**

| Compression | Original ResNet-18 | Fine-tuned (Ours) | Improvement |
|---|---|---|---|
| Low (2:1) | ~0.03 drop | ~0.01 drop | ↑ +0.02 |
| Medium (5:1) | ~0.10 drop | ~0.05 drop | ↑ +0.05 |
| High (10:1) | ~0.22 drop | ~0.14 drop | ↑ +0.08 |

Conclusion: Training ResNet-18 on wavelet-compressed images significantly improves its robustness under compression, confirming that domain-matched training data is an effective strategy for compression-resilient classification.

3

## KEY FINDINGS

- Fine-tuned model shows reduced accuracy drop at all compression levels
- Training on compressed data teaches robust feature extraction
- Small baseline accuracy trade-off — expected domain shift effect

04 SETUP

# Fine-tuning Model Two

## Approach & Hyperparameters

### Wavelet Compression

**Wavelet family:** Haar (db1)

**Decomposition level:** 2

**Thresholding:** Hard threshold

**LL band:** Always preserved

**Training threshold:** 0.15 (~5:1 ratio)

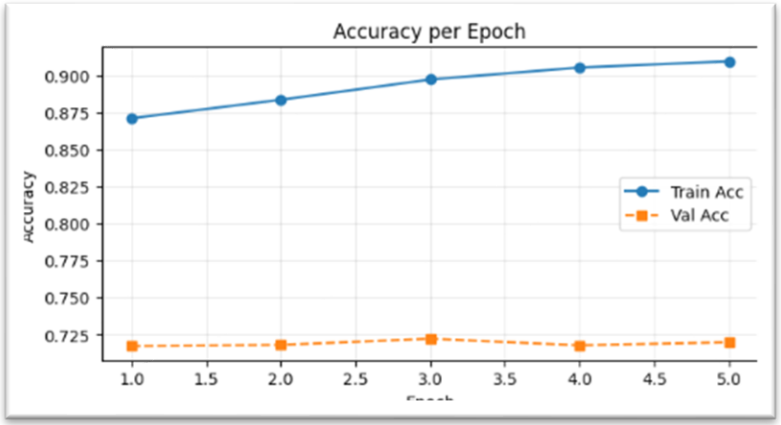
**Rationale:** *Balanced — not too aggressive, not too easy for the model*

### Fine-tuning Hyperparameters

| | |
|---|---|
| **Epochs:** | 5 |
| **Learning Rate:** | 5e-5 (↓ from 1e-4 in Part 1) |
| **Weight Decay:** | 1e-4 |
| **Batch Size:** | 32 |
| **Optimizer:** | AdamW |
| **Scheduler:** | ReduceLROnPlateau |
| **Early Stop:** | Patience = 3 |
| **Val Set:** | Uncompressed (fair comparison) |

# Training Results — MobileNetV3 on Compressed Data

| Epoch | Train Loss | Train Acc | Val Acc | LR |
|---|---|---|---|---|
| 1 | 0.4193 | 0.8709 | 0.7169 | 5e-5 |
| 2 | 0.3702 | 0.8834 | 0.7177 | 5e-5 |
| 3 | 0.3353 | 0.8971 | **0.7219** | 5e-5 |
| 4 | 0.3080 | 0.9052 | 0.7174 | 5e-5 |
| 5 | 0.2882 | 0.9094 | 0.7195 | 2.5e-5 |




**72.19%**
Best Val Acc
(Epoch 3)

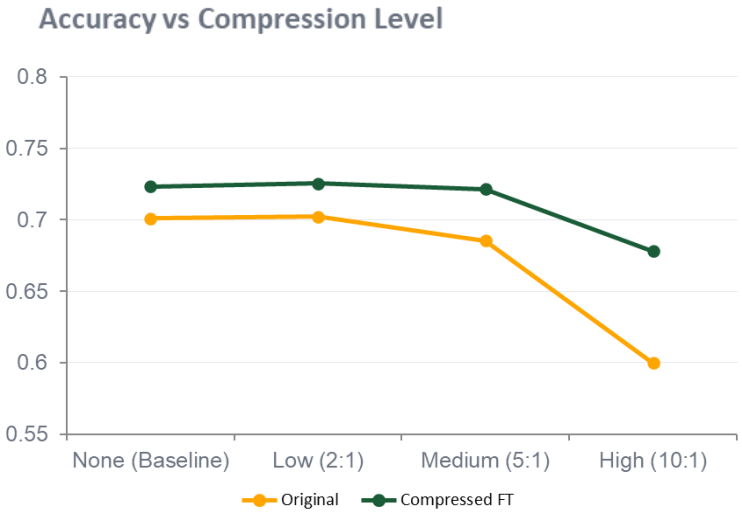
**90.94%**
Final Train Acc
(Epoch 5)

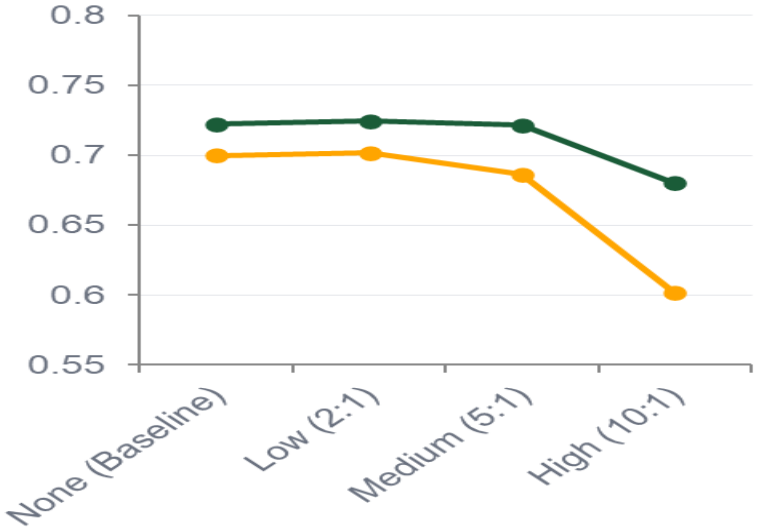
**0.2882**
Final
Train Loss

# Performance Comparison — Original vs Fine-tuned

| Model | Compression | Accuracy | Macro F1 |
|---|---|---|---|
| MobileNetV3 (Original) | None (Baseline) | 0.7010 | 0.6999 |
| MobileNetV3 (Original) | Low (2:1) | 0.7023 | 0.7017 |
| MobileNetV3 (Original) | Medium (5:1) | 0.6854 | 0.6864 |
| MobileNetV3 (Original) | High (10:1) | 0.5997 | 0.6014 |
| MobileNetV3 (Compressed FT) | None (Baseline) | 0.7234 | 0.7221 |
| MobileNetV3 (Compressed FT) | Low (2:1) | 0.7255 | 0.7243 |
| MobileNetV3 (Compressed FT) | Medium (5:1) | 0.7214 | 0.7214 |
| MobileNetV3 (Compressed FT) | High (10:1) | 0.6781 | 0.6799 |

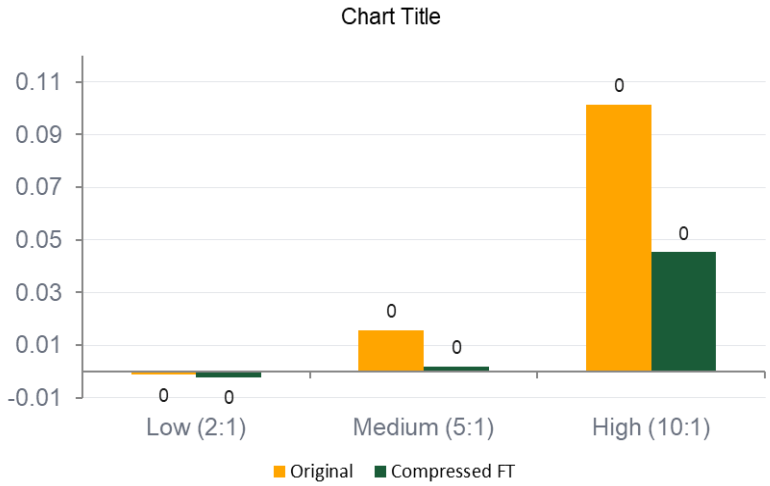
# Does Fine-tuning Help Generalization?

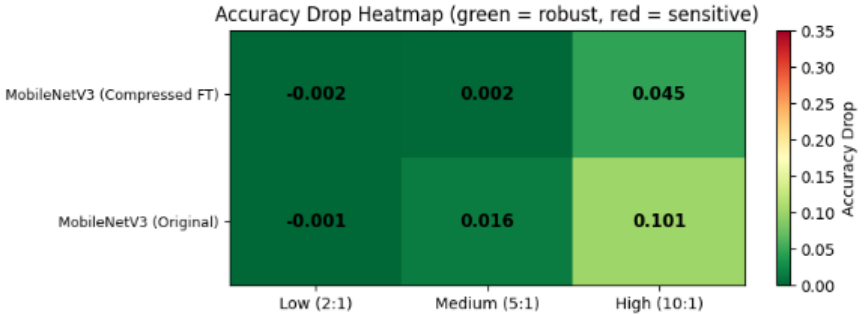



**Key Finding: The fine-tuned model consistently outperforms the original at ALL compression levels, especially at High (10:1): +7.84% accuracy gain**

# Robustness Analvsis — Accuracy Drop Under npression




| Model | Compression | Drop |
|---|---|---|
| Original | Low (2:1) | -0.001 |
| Original | Medium (5:1) | 0.016 |
| Original | High (10:1) | **0.101** |
| Compressed FT | Low (2:1) | -0.002 |
| Compressed FT | Medium (5:1) | **0.002** |
| Compressed FT | High (10:1) | **0.045** |

**55%** Less accuracy drop at High compression (0.045 vs 0.101)

**10×** Less degradation at Medium compression (0.002 vs 0.016)

# Observations Fine-tuning Model Two

01

### Compression Level

Medium compression (threshold=0.15, ~5:1 ratio) was chosen for training — balanced between information loss and robustness benefit.

02

### Hyperparameter Change

Learning rate reduced from 1e-4 → 5e-5 vs Part 1. Model is already pre-trained; smaller steps prevent overwriting learned features.

03

### Robustness Improved

Fine-tuned model drops only 4.5% at High (10:1) compression vs 10.1% for the original — a 55% reduction in accuracy drop.

04

### Better Generalization

Compressed FT model outperforms the original at ALL compression levels. Training on compressed data acts as effective data augmentation.

# References


[1] PyTorch, “ResNet18,” *Torchvision Documentation*. https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html

[2] PyTorch, “MobileNet V3,” *Torchvision Documentation*. https://docs.pytorch.org/vision/main/models/mobilenetv3.html

# THANKS!